

Dual Source Latent Demand Modeling for AI Search

Algorithms with Full Dry-Run Calculations

Gravton Labs — Parinaya Chaturvedi [parinayac@gravton.ai]

February 5, 2026

Abstract

This document provides two complete, self-contained estimation procedures: (i) Case 1 uses only an AI-search sensor (ASV) and performs Bayesian inference on the log-scale, then aggregates keyword-level demand to prompt-level demand via similarity-based mixture weights; (ii) Case 2 fuses ASV with classic search volume (SV) using an explicit coupling prior and closed-form Gaussian updates. Each case is written as a single section containing micro-steps, and each micro-step includes motivation, theory, parameter types (Observed/Calculated/Hyperparameters), explicit intermediate calculations, and a running numerical dry-run example.

Contents

1	Mathematical and Statistical Prerequisites	3
1.1	What problem are we solving (in plain math words)?	3
1.2	Notation and types of quantities	3
1.3	Log transform and why it is used	4
1.4	Random variables and probability distributions used	4
1.4.1	Gaussian / Normal distribution	4
1.4.2	Lognormal distribution	4
1.5	Core Bayesian inference concepts	5
1.5.1	Bayes' Rule	5
1.5.2	Conjugacy in the Gaussian model	5
1.6	Likelihood models for sensors	5
1.6.1	Single-sensor Gaussian observation (Case 1)	5
1.6.2	Two-sensor model (Case 2) and coupling	5
1.7	Weighted aggregation and similarity	6
1.8	Estimation methods referenced by the algorithm	6
1.8.1	MAP vs posterior mean	6
1.9	Error propagation and uncertainty reporting	6
1.10	Checklist: after reading this section, you should be able to	7
2	Mathematical and Statistical Derivations	7
2.1	Derivation A: Case 1 posterior for one keyword with ASV only	7
2.1.1	Model definition (single keyword k)	7
2.1.2	Posterior derivation using “complete the square”	7
2.2	Derivation B: Converting log-posterior to demand on original scale	8
2.3	Derivation C: Similarity weights and why softmax normalization appears	8
2.4	Derivation D: Aggregation on log-scale vs original scale	9
2.4.1	Option 1: Aggregate in original space	9
2.4.2	Option 2: Aggregate in log-space	9

2.5	Derivation A-D (a) : Empirical Estimation of Class- c Hyperparameters for Case 1 (ASV-only) [using above derivations]	9
2.5.1	D.1 Data required (observables)	10
2.5.2	D.2 Estimating the ASV log-bias b_A (or $b_{A,c}$)	10
2.5.3	D.3 Estimating sensor measurement noise $\sigma_{A,c}^2$ from the ASV time series	10
2.5.4	D.4 Estimating the class prior (μ_c, τ_c^2) (Empirical Bayes, explicit)	11
2.5.5	D.5 Plug-in to Case 1 posterior (no algorithm change)	12
2.6	Derivation E: Case 2 Fusion of ASV and SV as Two Gaussian Measurements	12
2.6.1	E.1 Motivation and modeling assumptions	12
2.6.2	E.2 Model specification (prior + likelihoods)	12
2.6.3	E.3 Posterior derivation from first principles (no “magic steps”)	13
2.6.4	E.4 Estimating the offset δ (when not fixed)	15
2.7	Derivation F: Coupling-Prior Variant (Two Latents with a Soft Constraint)	15
2.7.1	F.1 Model and meaning of the coupling prior	15
2.7.2	F.2 Deriving the MAP / posterior mean via quadratic optimization	16
2.7.3	F.2.0 Where does the negative log-posterior come from? What does “up to constants” mean?	17
2.7.4	F.3 Relationship between Derivation E and Derivation F	19
2.7.5	F.4 Practical guidance: how to choose / estimate σ_C^2 and δ	19
2.8	Derivation G: From keyword-level posteriors to prompt-level posterior (uncertainty)	20
2.9	Derivation H: Why “observed vs calculated vs hyperparameters” matters mathematically	20
2.10	End goal: after reading derivations, you should be able to re-derive every formula in Case 1 and Case 2	20
3	Case 1 (Only ASV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)	21

1. Mathematical and Statistical Prerequisites

1.1 What problem are we solving (in plain math words)?

We observe *noisy proxy sensors* that correlate with an unobserved quantity we truly care about:

latent monthly demand θ for a prompt/topic/intent.

Instead of observing θ directly, we observe:

- **ASV** (AI-search sensor value) for keywords, denoted $ASV(k)$.
- (Case 2 only) **SV** (classic search volume) for keywords, denoted $SV(k)$.

The core modeling idea is:

$$\underbrace{\log(\text{sensor})}_{\text{observed}} \approx \underbrace{\log(\text{latent demand})}_{\text{unknown}} + \underbrace{\text{noise}}_{\text{random}}.$$

Working on the log-scale turns multiplicative noise into additive noise, and makes positive quantities easier to model.

1.2 Notation and types of quantities

We use three **types** of quantities throughout Case 1 and Case 2.

Observed (O)

These are measured or externally provided:

- Keywords $k \in \mathcal{K}$.
- Observed AI-search sensor values $ASV(k)$.
- (Case 2) Observed search volumes $SV(k)$.
- Text embeddings (or similarity scores) used to relate keyword k to prompt/topic p .

Calculated (C)

These are computed inside the algorithm:

- Log-sensor values: $y_A(k) = \log ASV(k)$, $y_S(k) = \log SV(k)$.
- Keyword-level posterior parameters: means and variances for latent log-demand.
- Similarity weights and normalized mixture weights.
- Prompt-level aggregated latent demand.

Hyperparameters (H)

These are modeling choices you set:

- Prior mean and variance for latent log-demand.
- Noise variances of sensors on the log-scale (ASV noise, SV noise).
- Coupling strength between ASV and SV latent signals (Case 2).
- Similarity temperature or scaling, smoothing constants, and any regularization.

1.3 Log transform and why it is used

If a quantity $X > 0$ varies multiplicatively (common in volumes),

$$X = \theta \cdot \varepsilon,$$

where $\varepsilon > 0$ is multiplicative noise, then taking logs yields

$$\log X = \log \theta + \log \varepsilon.$$

If $\log \varepsilon$ is approximately normal, we get a linear Gaussian model, which leads to:

- closed-form Bayesian updates (no sampling required),
- numerically stable computations,
- interpretability: additive decompositions on the log-scale correspond to multiplicative decompositions on the original scale.

1.4 Random variables and probability distributions used

1.4.1 Gaussian / Normal distribution

A random variable Z is normal:

$$Z \sim \mathcal{N}(\mu, \sigma^2)$$

with density

$$p(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z - \mu)^2}{2\sigma^2}\right).$$

Key facts:

- Sum of independent normals is normal.
- Posterior in a linear-Gaussian model remains Gaussian (conjugacy).

1.4.2 Lognormal distribution

If

$$Z \sim \mathcal{N}(\mu, \sigma^2),$$

then

$$X = \exp(Z) \sim \text{LogNormal}(\mu, \sigma^2).$$

Useful identities:

$$\mathbb{E}[X] = \exp\left(\mu + \frac{\sigma^2}{2}\right), \quad \text{Median}(X) = \exp(\mu), \quad \text{Mode}(X) = \exp(\mu - \sigma^2).$$

This matters because latent demand is positive. If we infer $Z = \log \theta$ as Gaussian, then $\theta = \exp(Z)$ is lognormal.

1.5 Core Bayesian inference concepts

1.5.1 Bayes' Rule

For parameter θ and observed data D :

$$p(\theta | D) = \frac{p(D | \theta) p(\theta)}{p(D)}.$$

Interpretation:

- $p(\theta)$: **prior** belief before seeing data.
- $p(D | \theta)$: **likelihood** how data is generated from θ .
- $p(\theta | D)$: **posterior** belief after seeing data.

1.5.2 Conjugacy in the Gaussian model

If the prior is Gaussian and the likelihood is Gaussian with linear relationship, the posterior is Gaussian with *closed-form* formulas. This is why Case 1 and Case 2 are designed to be linear-Gaussian on the log-scale.

Note: A Linear Gaussian model (or system) defines a relationship where variables follow a Gaussian (normal) distribution, and the mean of the dependent variable is a linear function of its parent variables, plus Gaussian noise. Commonly used in regression and Kalman filters, this structure allows for efficient, exact inference because linear transformations of Gaussian variables remain Gaussian. Structure: The model is generally expressed as $x = Az + \epsilon$, where x is the observed data, z is the latent/parent variable, A is a transformation matrix, and $\epsilon \sim N(0, \Sigma)$ is Gaussian noise.

1.6 Likelihood models for sensors

1.6.1 Single-sensor Gaussian observation (Case 1)

Let $z_k = \log \theta_k$ be latent log-demand for keyword k . We observe the log ASV sensor:

$$y_A(k) = \log ASV(k).$$

Model:

$$y_A(k) | z_k \sim \mathcal{N}(z_k, \sigma_A^2).$$

Here σ_A^2 is a hyperparameter: sensor noise on log-scale.

1.6.2 Two-sensor model (Case 2) and coupling

In Case 2 we have

$$y_A(k) = \log ASV(k), \quad y_S(k) = \log SV(k).$$

A common structure is:

$$y_A(k) | z_k \sim \mathcal{N}(z_k, \sigma_A^2), \quad y_S(k) | z_k \sim \mathcal{N}(z_k + \delta, \sigma_S^2),$$

where δ is a calibration offset capturing systematic differences between SV and ASV scales. Alternatively, we can use a **coupled latent** formulation with a coupling prior; the derivations for both appear in the derivations section.

1.7 Weighted aggregation and similarity

We often need to map keyword-level estimates to prompt/topic-level estimates. We use similarity-based weights:

$$s_{p,k} \in [0, 1] \quad (\text{similarity between prompt } p \text{ and keyword } k).$$

A common conversion to weights is a softmax / normalized exponential:

$$w_{p,k} = \frac{\exp(\tau s_{p,k})}{\sum_{j \in \mathcal{K}} \exp(\tau s_{p,j})},$$

where $\tau > 0$ is a hyperparameter (**temperature/scale**) controlling sharpness.

Aggregation examples:

- On **log-scale** (mixture of log-demands):

$$\hat{z}_p = \sum_k w_{p,k} \hat{z}_k.$$

- On **original scale** (mixture of demands):

$$\hat{\theta}_p = \sum_k w_{p,k} \hat{\theta}_k.$$

Important: averaging in log-space is not the same as averaging in original space. We will derive and interpret both choices in the derivations section.

1.8 Estimation methods referenced by the algorithm

1.8.1 MAP vs posterior mean

For Gaussian posteriors:

$$z_k \mid D \sim \mathcal{N}(\mu_k, v_k).$$

Then:

$$\text{posterior mean} = \mu_k, \quad \text{MAP} = \mu_k,$$

so they coincide. On original scale ($\theta_k = \exp(z_k)$), mean and median differ:

$$\text{Median}(\theta_k) = \exp(\mu_k), \quad \mathbb{E}[\theta_k] = \exp\left(\mu_k + \frac{v_k}{2}\right).$$

This is crucial when you convert back from log-space.

1.9 Error propagation and uncertainty reporting

If $z \sim \mathcal{N}(\mu, v)$ and $\theta = \exp(z)$, then θ is lognormal. A $(1 - \alpha)$ credible interval for z :

$$[\mu - z_{\alpha/2}\sqrt{v}, \mu + z_{\alpha/2}\sqrt{v}],$$

maps to a multiplicative interval for θ :

$$[\exp(\mu - z_{\alpha/2}\sqrt{v}), \exp(\mu + z_{\alpha/2}\sqrt{v})].$$

This is why uncertainty on log-scale translates into multiplicative uncertainty on demand.

1.10 Checklist: after reading this section, you should be able to

1. Explain why we use log-transform and Gaussian noise.
2. Write Bayes' rule and interpret prior/likelihood/posterior.
3. Derive (or follow) Gaussian conjugate updates.
4. Convert from Gaussian on log-scale to lognormal on original scale.
5. Understand similarity-based weighting and its hyperparameters.
6. Understand how uncertainty propagates through exponentiation.

2. Mathematical and Statistical Derivations

2.1 Derivation A: Case 1 posterior for one keyword with ASV only

2.1.1 Model definition (single keyword k)

Latent variable (unknown).

$$z_k \equiv \log \theta_k.$$

Prior (hyperparameters).

$$z_k \sim \mathcal{N}(\mu_0, v_0).$$

Observation (observed).

$$y_A(k) \equiv \log ASV(k).$$

Likelihood (hyperparameter σ_A^2).

$$y_A(k) \mid z_k \sim \mathcal{N}(z_k, \sigma_A^2).$$

2.1.2 Posterior derivation using “complete the square”

Bayes:

$$p(z_k \mid y_A) \propto p(y_A \mid z_k) p(z_k).$$

Write both densities up to proportionality:

$$p(y_A \mid z_k) \propto \exp\left(-\frac{(y_A - z_k)^2}{2\sigma_A^2}\right), \quad p(z_k) \propto \exp\left(-\frac{(z_k - \mu_0)^2}{2v_0}\right).$$

Multiply:

$$p(z_k \mid y_A) \propto \exp\left(-\frac{(y_A - z_k)^2}{2\sigma_A^2} - \frac{(z_k - \mu_0)^2}{2v_0}\right).$$

Expand the squares (showing all intermediate algebra):

$$(y_A - z_k)^2 = z_k^2 - 2y_A z_k + y_A^2, \quad (z_k - \mu_0)^2 = z_k^2 - 2\mu_0 z_k + \mu_0^2.$$

Substitute:

$$\begin{aligned} & -\frac{1}{2\sigma_A^2}(z_k^2 - 2y_A z_k + y_A^2) - \frac{1}{2v_0}(z_k^2 - 2\mu_0 z_k + \mu_0^2) \\ &= -\frac{1}{2}\left(\frac{1}{\sigma_A^2} + \frac{1}{v_0}\right)z_k^2 + \left(\frac{y_A}{\sigma_A^2} + \frac{\mu_0}{v_0}\right)z_k - \frac{1}{2}\left(\frac{y_A^2}{\sigma_A^2} + \frac{\mu_0^2}{v_0}\right). \end{aligned}$$

The last term does not depend on z_k and can be absorbed into the normalizing constant. So the posterior is proportional to:

$$\exp\left(-\frac{1}{2}az_k^2 + bz_k\right), \quad \text{where } a = \left(\frac{1}{\sigma_A^2} + \frac{1}{v_0}\right), \quad b = \left(\frac{y_A}{\sigma_A^2} + \frac{\mu_0}{v_0}\right).$$

Complete the square:

$$-\frac{1}{2}az_k^2 + bz_k = -\frac{1}{2}a\left(z_k^2 - 2\frac{b}{a}z_k\right) = -\frac{1}{2}a\left[\left(z_k - \frac{b}{a}\right)^2 - \left(\frac{b}{a}\right)^2\right].$$

Thus:

$$p(z_k | y_A) \propto \exp\left(-\frac{1}{2}a\left(z_k - \frac{b}{a}\right)^2\right),$$

which is a Gaussian with:

$$v_k = \frac{1}{a} = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2}\right)^{-1}, \quad \mu_k = \frac{b}{a} = v_k \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2}\right).$$

Interpretation (precision-weighted average). Define **precision** as $\pi = \frac{1}{\text{variance}}$. Then

$$\mu_k = \frac{\pi_0\mu_0 + \pi_A y_A}{\pi_0 + \pi_A}, \quad v_k = \frac{1}{\pi_0 + \pi_A},$$

where $\pi_0 = 1/v_0$ and $\pi_A = 1/\sigma_A^2$.

2.2 Derivation B: Converting log-posterior to demand on original scale

Given

$$z_k | D \sim \mathcal{N}(\mu_k, v_k), \quad \theta_k = \exp(z_k),$$

then θ_k is lognormal. Key outputs:

$$\text{Median}(\theta_k) = \exp(\mu_k), \quad \mathbb{E}[\theta_k] = \exp\left(\mu_k + \frac{v_k}{2}\right).$$

Which should the algorithm use?

- If you want a **typical** value robust to skew: use $\exp(\mu_k)$ (median).
- If you want the **expected** volume (mean): use $\exp(\mu_k + v_k/2)$.

Both are mathematically correct; they answer different questions. Your Case 1/Case 2 algorithms should state explicitly which one they output.

2.3 Derivation C: Similarity weights and why softmax normalization appears

Let $s_{p,k}$ be similarity between prompt/topic p and keyword k . We want nonnegative weights that sum to 1:

$$w_{p,k} \geq 0, \quad \sum_k w_{p,k} = 1.$$

A standard construction is:

$$w_{p,k} = \frac{\exp(\tau s_{p,k})}{\sum_j \exp(\tau s_{p,j})}.$$

Why exponentials? Exponentials convert additive differences in similarity into multiplicative ratios:

$$\frac{w_{p,k_1}}{w_{p,k_2}} = \exp(\tau(s_{p,k_1} - s_{p,k_2})).$$

So if s_{p,k_1} is bigger than s_{p,k_2} by Δ , then k_1 is weighted $\exp(\tau\Delta)$ times more than k_2 .

Role of τ (hyperparameter).

- Large $\tau \Rightarrow$ weights become *peaky* (nearly one-hot).
- Small $\tau \Rightarrow$ weights become *uniform*.

2.4 Derivation D: Aggregation on log-scale vs original scale

Suppose keyword-level latent demands are random variables $\theta_k = \exp(z_k)$. A prompt-level demand is often approximated by a weighted combination.

2.4.1 Option 1: Aggregate in original space

$$\theta_p \approx \sum_k w_{p,k} \theta_k.$$

If we use posterior mean estimates:

$$\hat{\theta}_p = \sum_k w_{p,k} \mathbb{E}[\theta_k | D] = \sum_k w_{p,k} \exp\left(\mu_k + \frac{v_k}{2}\right).$$

This has a direct “expected volume” interpretation.

2.4.2 Option 2: Aggregate in log-space

$$z_p \approx \sum_k w_{p,k} z_k, \quad \Rightarrow \quad \hat{z}_p = \sum_k w_{p,k} \mu_k.$$

If you then output a typical demand:

$$\hat{\theta}_p = \exp(\hat{z}_p),$$

this corresponds to a kind of *geometric averaging*.

Important warning. In general,

$$\exp\left(\sum_k w_k \mu_k\right) \neq \sum_k w_k \exp(\mu_k).$$

So you must choose the aggregation domain intentionally and keep it consistent with what you claim to estimate.

For our Case 1 and 2, we will be using Option 1, but this is subject to experiments.

2.5 Derivation A-D (a) : Empirical Estimation of Class- c Hyperparameters for Case 1 (ASV-only) [using above derivations]

Goal. Case 1 uses class-conditioned hyperparameters

$$(\mu_c, \tau_c^2, b_A, \sigma_{A,c}^2)$$

in the Bayesian model

$$a_k \sim \mathcal{N}(\mu_c, \tau_c^2), \quad x_{k,t} | a_{k,t} \sim \mathcal{N}(a_{k,t} + b_A, \sigma_{A,c}^2), \quad x_{k,t} \triangleq \log(\text{ASV}_{k,t}).$$

This derivation shows how to compute these quantities explicitly from historical sensor time series, without assuming values.

Why classes exist (hierarchical pooling). The class index $c = c(k)$ defines pooling groups

$$K_c \triangleq \{k : c(k) = c\}, \quad |K_c| = n_c,$$

so that keywords in the same intent/semantic family share statistical properties:

- prior center and dispersion (μ_c, τ_c^2) (between-keyword variability in true demand),
- sensor measurement noise $\sigma_{A,c}^2$ (how noisy ASV is for this intent class),
- and optionally sensor bias b_A (systematic log-offset).

This is the mechanism that induces hierarchical shrinkage in the posterior.

2.5.1 D.1 Data required (observables)

Let $t = 1, \dots, T$ index months (or other fixed intervals). For each keyword $k \in K_c$, observe the ASV sensor:

$$\text{ASV}_{k,t} \quad (\text{Observed}), \quad x_{k,t} \triangleq \log(\text{ASV}_{k,t}) \quad (\text{Calculated}).$$

2.5.2 D.2 Estimating the ASV log-bias b_A (or $b_{A,c}$)

Model identity. From

$$x_{k,t} \approx a_{k,t} + b_A + \varepsilon_{k,t}, \quad \varepsilon_{k,t} \sim \mathcal{N}(0, \sigma_{A,c}^2),$$

bias is the systematic difference between the sensor and a truth-aligned quantity. Therefore, to estimate b_A we need a calibration proxy $g_{k,t}$ such that

$$g_{k,t} \approx a_{k,t} = \log(A_{k,t}^*) \quad \text{for a calibration subset of keyword-time pairs.}$$

Calibration set. Let $\mathcal{C} \subseteq \{(k, t)\}$ denote the set of calibration pairs with such a proxy, and let $|\mathcal{C}| = N_{\mathcal{C}}$.

Bias estimator (explicit). Using $x_{k,t} - g_{k,t} \approx b_A$, the unbiased estimator is

$$\hat{b}_A = \frac{1}{N_{\mathcal{C}}} \sum_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

Robust alternative:

$$\hat{b}_A^{\text{robust}} = \text{median}_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

If bias differs by class, estimate within class:

$$\hat{b}_{A,c} = \frac{1}{N_{\mathcal{C},c}} \sum_{(k,t) \in \mathcal{C} \cap (K_c \times \{1, \dots, T\})} (x_{k,t} - g_{k,t}).$$

2.5.3 D.3 Estimating sensor measurement noise $\sigma_{A,c}^2$ from the ASV time series

Baseline + residual principle. To estimate measurement noise, compute deviations around an estimate of the keyword's underlying level.

Per-keyword baseline (explicit). The simplest baseline is the time-mean:

$$\bar{x}_k \triangleq \frac{1}{T} \sum_{t=1}^T x_{k,t}.$$

Then compute the per-keyword residual variance:

$$\hat{\sigma}_A^2(k) \triangleq \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2.$$

Pooling to the class level (explicit). Mean pooling:

$$\hat{\sigma}_{A,c}^2 \triangleq \frac{1}{n_c} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

Robust pooling:

$$\hat{\sigma}_{A,c,\text{robust}}^2 \triangleq \text{median}_{k \in K_c} \hat{\sigma}_A^2(k).$$

If drift/seasonality exists (still explicit). If the latent $a_{k,t}$ changes materially over the window, residuals around $\bar{x}_k$ include drift. A principled fix is detrending via a smoothing operator:

$$\hat{m}_{k,t} = \text{Smooth}(x_{k,1:T})_t, \quad r_{k,t} = x_{k,t} - \hat{m}_{k,t},$$

then

$$\hat{\sigma}_A^2(k) = \frac{1}{T-1} \sum_{t=1}^T r_{k,t}^2, \quad \hat{\sigma}_{A,c}^2 = \frac{1}{n_c} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

2.5.4 D.4 Estimating the class prior (μ_c, τ_c^2) (Empirical Bayes, explicit)

Keyword-level level estimate (explicit). First de-bias and average to estimate each keyword's latent log-level:

$$\hat{a}_k \triangleq \frac{1}{T} \sum_{t=1}^T (x_{k,t} - \hat{b}_A) = \bar{x}_k - \hat{b}_A.$$

This is a noisy estimate of the keyword's latent mean level.

Class prior mean (explicit).

$$\hat{\mu}_c \triangleq \frac{1}{n_c} \sum_{k \in K_c} \hat{a}_k.$$

Robust alternative:

$$\hat{\mu}_c^{\text{robust}} = \text{median}_{k \in K_c} \hat{a}_k.$$

Class between-keyword variance (explicit with noise removal). Across keywords, the sample variance of $\hat{a}_k$ mixes true between-keyword variance τ_c^2 and estimation noise from averaging T measurements. Since

$$\hat{a}_k = \bar{x}_k - \hat{b}_A, \quad \text{Var}(\bar{x}_k) \approx \frac{\sigma_{A,c}^2}{T},$$

we have approximately

$$\text{Var}(\hat{a}_k) \approx \tau_c^2 + \frac{\sigma_{A,c}^2}{T}.$$

Hence an explicit Empirical-Bayes estimator is

$$\hat{\tau}_c^2 \triangleq \max \left\{ \underbrace{\frac{1}{n_c - 1} \sum_{k \in K_c} (\hat{a}_k - \hat{\mu}_c)^2}_{\text{sample variance across keywords}} - \underbrace{\frac{\hat{\sigma}_{A,c}^2}{T}}_{\text{remove averaging noise}}, \epsilon \right\},$$

where $\epsilon > 0$ is a small variance floor to ensure non-negativity.

2.5.5 D.5 Plug-in to Case 1 posterior (no algorithm change)

Once

$$(\hat{\mu}_c, \hat{\tau}_c^2, \hat{b}_A, \hat{\sigma}_{A,c}^2)$$

are computed, Case 1 uses exactly the same closed-form Normal–Normal posterior:

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\hat{\tau}_c^2} + \frac{1}{\hat{\sigma}_{A,c}^2} \right)^{-1}, \quad \mu_{k,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\hat{\mu}_c}{\hat{\tau}_c^2} + \frac{x_k - \hat{b}_A}{\hat{\sigma}_{A,c}^2} \right),$$

followed by lognormal back-transform and prompt-level mixing as already specified in Case 1.

2.6 Derivation E: Case 2 Fusion of ASV and SV as Two Gaussian Measurements

This derivation is the canonical *conjugate Gaussian update* when we have *two conditionally independent noisy measurements* of the same latent quantity. We will re-derive every step (including why “precisions add”) and also interpret the result.

2.6.1 E.1 Motivation and modeling assumptions

Goal. For a keyword k , we want to estimate the latent *true* log-demand

$$z_k \triangleq \log \theta_k,$$

using two observed sensors:

$$y_A \triangleq \log(\text{ASV}(k)), \quad y_S \triangleq \log(\text{SV}(k)).$$

Key assumptions.

1. **Latent truth exists.** There is a single underlying true log-demand z_k .
2. **Gaussian sensor noise in log-space.** Each sensor equals truth plus noise (in log scale).
3. **Conditional independence given truth.** Once z_k is fixed, the two sensors’ noises are independent:

$$p(y_A, y_S \mid z_k) = p(y_A \mid z_k) p(y_S \mid z_k).$$

This is the only assumption needed for the “precision additivity” result.

2.6.2 E.2 Model specification (prior + likelihoods)

Prior (before seeing sensors). We assume a Gaussian prior for the latent log-demand:

$$z_k \sim \mathcal{N}(\mu_0, v_0),$$

where:

- μ_0 is the prior mean (could be global or class-conditioned; here we write a single prior for clarity).
- v_0 is the prior variance (uncertainty before observing sensors).

ASV likelihood. We model the ASV log-sensor as:

$$y_A \mid z_k \sim \mathcal{N}(z_k, \sigma_A^2),$$

meaning:

$$y_A = z_k + \varepsilon_A, \quad \varepsilon_A \sim \mathcal{N}(0, \sigma_A^2).$$

SV likelihood with calibration offset. In practice SV often differs from ASV by a systematic log-scale offset δ (a multiplicative factor in linear scale). We model:

$$y_S \mid z_k \sim \mathcal{N}(z_k + \delta, \sigma_S^2),$$

meaning:

$$y_S = z_k + \delta + \varepsilon_S, \quad \varepsilon_S \sim \mathcal{N}(0, \sigma_S^2).$$

Aligned SV observation (removing bias). Define the bias-corrected SV log-observation:

$$\tilde{y}_S \triangleq y_S - \delta.$$

Then:

$$\tilde{y}_S \mid z_k \sim \mathcal{N}(z_k, \sigma_S^2),$$

so now both sensors are centered at the same latent z_k .

2.6.3 E.3 Posterior derivation from first principles (no “magic steps”)

Posterior definition. By Bayes’ rule:

$$p(z_k \mid y_A, \tilde{y}_S) \propto p(z_k) p(y_A \mid z_k) p(\tilde{y}_S \mid z_k).$$

Each term is Gaussian in z_k , so the product is proportional to a Gaussian in z_k . We will show this by expanding the *log posterior* and completing the square.

Write each density up to proportionality in z_k . Ignoring constants that do not depend on z_k :

$$\begin{aligned} p(z_k) &\propto \exp\left(-\frac{1}{2v_0}(z_k - \mu_0)^2\right), \\ p(y_A \mid z_k) &\propto \exp\left(-\frac{1}{2\sigma_A^2}(y_A - z_k)^2\right), \\ p(\tilde{y}_S \mid z_k) &\propto \exp\left(-\frac{1}{2\sigma_S^2}(\tilde{y}_S - z_k)^2\right). \end{aligned}$$

Multiply them (add exponents). Thus:

$$p(z_k \mid y_A, \tilde{y}_S) \propto \exp\left(-\frac{1}{2}\left[\frac{(z_k - \mu_0)^2}{v_0} + \frac{(y_A - z_k)^2}{\sigma_A^2} + \frac{(\tilde{y}_S - z_k)^2}{\sigma_S^2}\right]\right).$$

Define the negative log-posterior (up to constants):

$$\mathcal{L}(z_k) \triangleq \frac{(z_k - \mu_0)^2}{v_0} + \frac{(y_A - z_k)^2}{\sigma_A^2} + \frac{(\tilde{y}_S - z_k)^2}{\sigma_S^2}.$$

Expand $\mathcal{L}(z_k)$ into a quadratic in z_k . Expand each square:

$$(z_k - \mu_0)^2 = z_k^2 - 2\mu_0 z_k + \mu_0^2,$$

$$(y_A - z_k)^2 = z_k^2 - 2y_A z_k + y_A^2,$$

$$(\tilde{y}_S - z_k)^2 = z_k^2 - 2\tilde{y}_S z_k + \tilde{y}_S^2.$$

Substitute:

$$\mathcal{L}(z_k) = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \right) z_k^2 - 2 \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \right) z_k + (\text{constants}).$$

Let

$$\pi_k \triangleq \frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \quad (\text{posterior precision}),$$

$$b_k \triangleq \frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \quad (\text{precision-weighted sum}).$$

Then:

$$\mathcal{L}(z_k) = \pi_k z_k^2 - 2b_k z_k + (\text{constants}).$$

Complete the square to identify the Gaussian posterior. We rewrite:

$$\pi_k z_k^2 - 2b_k z_k = \pi_k \left[z_k^2 - 2 \left(\frac{b_k}{\pi_k} \right) z_k \right] = \pi_k \left[\left(z_k - \frac{b_k}{\pi_k} \right)^2 - \left(\frac{b_k}{\pi_k} \right)^2 \right].$$

Thus (up to constants):

$$p(z_k \mid y_A, \tilde{y}_S) \propto \exp \left(-\frac{\pi_k}{2} \left(z_k - \frac{b_k}{\pi_k} \right)^2 \right),$$

which is exactly a Gaussian with:

$$v_k = \frac{1}{\pi_k}, \quad \mu_k = \frac{b_k}{\pi_k}.$$

Substituting b_k back:

$$v_k = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \right)^{-1}$$

$$\mu_k = v_k \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \right).$$

Why “precisions add”? (Statistical intuition) Each Gaussian term contributes a quadratic penalty in z_k :

$$\frac{(z_k - \text{center})^2}{\text{variance}}.$$

Quadratic penalties add; therefore the coefficients of z_k^2 add. Those coefficients are precisely the *precisions* (1/variance). Hence precision additivity is just “adding curvature” of independent quadratic losses.

Interpretation as a weighted average. Define weights proportional to precision:

$$w_0 = \frac{1/v_0}{\pi_k}, \quad w_A = \frac{1/\sigma_A^2}{\pi_k}, \quad w_S = \frac{1/\sigma_S^2}{\pi_k}.$$

Then $w_0 + w_A + w_S = 1$ and:

$$\mu_k = w_0\mu_0 + w_A y_A + w_S \tilde{y}_S.$$

So the posterior mean is a convex combination of prior mean and both sensors, where more reliable sources (smaller variance) get larger weights.

2.6.4 E.4 Estimating the offset δ (when not fixed)

Meaning of δ . Because we work in log space,

$$y_S = \log(\text{SV}) \approx z_k + \delta \iff \text{SV} \approx e^\delta \cdot \theta_k.$$

So e^δ is a multiplicative bias factor of SV relative to the latent.

Estimator from a calibration set. Suppose we have a calibration set $\mathcal{C}$ of keywords where the two sensors are believed to track the same latent on average. We model:

$$y_S(k) - y_A(k) = \delta + \eta_k, \quad \eta_k \text{ zero-mean noise.}$$

Then the MLE / least squares estimate of δ is the sample mean:

$$\hat{\delta} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).$$

Derivation (one line). Minimize:

$$\hat{\delta} = \arg \min_{\delta} \sum_{k \in \mathcal{C}} \left((y_S(k) - y_A(k)) - \delta \right)^2,$$

differentiate and set to zero:

$$-2 \sum_{k \in \mathcal{C}} \left((y_S(k) - y_A(k)) - \delta \right) = 0 \Rightarrow \delta = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).$$

Robust alternative (median). If there are outliers, use:

$$\hat{\delta}_{\text{med}} = \text{median}\{y_S(k) - y_A(k) : k \in \mathcal{C}\}.$$

2.7 Derivation F: Coupling-Prior Variant (Two Latents with a Soft Constraint)

Derivation E assumes there is one latent z_k that both sensors measure (after offset correction). Derivation F is used when you want to explicitly represent that “AI-demand” and “classic-demand” may be *different latent constructs* but *usually close*, and you want a controllable “soft tie” between them.

2.7.1 F.1 Model and meaning of the coupling prior

Latents.

$$z_{A,k} \triangleq \log \theta_{A,k} \quad (\text{AI-demand latent}), \quad z_{S,k} \triangleq \log \theta_{S,k} \quad (\text{classic-demand latent}).$$

Observations.

$$y_A \mid z_{A,k} \sim \mathcal{N}(z_{A,k}, \sigma_A^2), \quad y_S \mid z_{S,k} \sim \mathcal{N}(z_{S,k}, \sigma_S^2).$$

Coupling prior (soft constraint). We encode that $z_{S,k}$ should be close to $z_{A,k}$ up to an offset δ :

$$z_{S,k} - z_{A,k} \sim \mathcal{N}(\delta, \sigma_C^2).$$

Equivalently:

$$z_{S,k} = z_{A,k} + \delta + \epsilon_C, \quad \epsilon_C \sim \mathcal{N}(0, \sigma_C^2).$$

Interpretation of σ_C^2 .

- Small σ_C^2 : strong coupling; forces $z_{S,k} \approx z_{A,k} + \delta$.
- Large σ_C^2 : weak coupling; the latents can diverge; sensors are almost independent.

Optional priors on the two latents. To fully specify a posterior, we commonly put Gaussian priors:

$$z_{A,k} \sim \mathcal{N}(\mu_A, v_A), \quad z_{S,k} \sim \mathcal{N}(\mu_S, v_S).$$

(If you omit these, the posterior can still be proper if the likelihoods and coupling are sufficient, but in practice priors stabilize inference.)

2.7.2 F.2 Deriving the MAP / posterior mean via quadratic optimization

Because all terms are Gaussian, the joint posterior is multivariate Gaussian. The cleanest way to see this (and to compute the posterior mean) is to write the *negative log-posterior* as a sum of squared terms, then solve the resulting linear system (normal equations).

Negative log-posterior (up to constants). Let $z_A \equiv z_{A,k}$ and $z_S \equiv z_{S,k}$ for readability. Then:

$$\mathcal{J}(z_A, z_S) = \underbrace{\frac{(z_A - \mu_A)^2}{v_A}}_{\text{prior on } z_A} + \underbrace{\frac{(z_S - \mu_S)^2}{v_S}}_{\text{prior on } z_S} + \underbrace{\frac{(y_A - z_A)^2}{\sigma_A^2}}_{\text{ASV likelihood}} + \underbrace{\frac{(y_S - z_S)^2}{\sigma_S^2}}_{\text{SV likelihood}} + \underbrace{\frac{((z_S - z_A) - \delta)^2}{\sigma_C^2}}_{\text{coupling}}.$$

This is a quadratic form in (z_A, z_S) .

Take partial derivatives and set to zero. Compute $\partial \mathcal{J} / \partial z_A$ and $\partial \mathcal{J} / \partial z_S$.

First, note:

$$\frac{\partial}{\partial z_A} \frac{(z_A - \mu_A)^2}{v_A} = \frac{2(z_A - \mu_A)}{v_A}, \quad \frac{\partial}{\partial z_A} \frac{(y_A - z_A)^2}{\sigma_A^2} = \frac{2(z_A - y_A)}{\sigma_A^2},$$

and for coupling term:

$$\frac{\partial}{\partial z_A} \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} = \frac{2}{\sigma_C^2} ((z_S - z_A) - \delta) \cdot (-1) = \frac{2}{\sigma_C^2} (z_A - z_S + \delta).$$

Thus:

$$\frac{\partial \mathcal{J}}{\partial z_A} = \frac{2(z_A - \mu_A)}{v_A} + \frac{2(z_A - y_A)}{\sigma_A^2} + \frac{2(z_A - z_S + \delta)}{\sigma_C^2}.$$

Set to zero and divide by 2:

$$\frac{z_A - \mu_A}{v_A} + \frac{z_A - y_A}{\sigma_A^2} + \frac{z_A - z_S + \delta}{\sigma_C^2} = 0.$$

Rearrange:

$$\left(\frac{1}{v_A} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_C^2}\right) z_A - \frac{1}{\sigma_C^2} z_S = \frac{\mu_A}{v_A} + \frac{y_A}{\sigma_A^2} - \frac{\delta}{\sigma_C^2}.$$

Similarly for z_S :

$$\begin{aligned} \frac{\partial}{\partial z_S} \frac{(z_S - \mu_S)^2}{v_S} &= \frac{2(z_S - \mu_S)}{v_S}, & \frac{\partial}{\partial z_S} \frac{(y_S - z_S)^2}{\sigma_S^2} &= \frac{2(z_S - y_S)}{\sigma_S^2}, \\ \frac{\partial}{\partial z_S} \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} &= \frac{2}{\sigma_C^2} ((z_S - z_A) - \delta). \end{aligned}$$

So:

$$\begin{aligned} \frac{z_S - \mu_S}{v_S} + \frac{z_S - y_S}{\sigma_S^2} + \frac{(z_S - z_A) - \delta}{\sigma_C^2} &= 0, \\ -\frac{1}{\sigma_C^2} z_A + \left(\frac{1}{v_S} + \frac{1}{\sigma_S^2} + \frac{1}{\sigma_C^2}\right) z_S &= \frac{\mu_S}{v_S} + \frac{y_S}{\sigma_S^2} + \frac{\delta}{\sigma_C^2}. \end{aligned}$$

Write as a linear system. Define the unknown vector $\mathbf{z} = [z_A \ z_S]^\top$. Then:

$$\underbrace{\begin{pmatrix} \frac{1}{v_A} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_C^2} & -\frac{1}{\sigma_C^2} \\ -\frac{1}{\sigma_C^2} & \frac{1}{v_S} + \frac{1}{\sigma_S^2} + \frac{1}{\sigma_C^2} \end{pmatrix}}_{\triangleq \mathbf{Q} \text{ (posterior precision matrix)}} \begin{pmatrix} z_A \\ z_S \end{pmatrix} = \underbrace{\begin{pmatrix} \frac{\mu_A}{v_A} + \frac{y_A}{\sigma_A^2} - \frac{\delta}{\sigma_C^2} \\ \frac{\mu_S}{v_S} + \frac{y_S}{\sigma_S^2} + \frac{\delta}{\sigma_C^2} \end{pmatrix}}_{\triangleq \mathbf{h}}.$$

Hence the posterior mean (and MAP, since Gaussian) is:

$$\boxed{\mathbb{E}[\mathbf{z} \mid y_A, y_S] = \mathbf{Q}^{-1} \mathbf{h}.}$$

Optional explicit inversion (so nothing is hidden). For a 2×2 matrix,

$$\mathbf{Q}^{-1} = \frac{1}{\det(\mathbf{Q})} \begin{pmatrix} Q_{22} & -Q_{12} \\ -Q_{21} & Q_{11} \end{pmatrix}, \quad \det(\mathbf{Q}) = Q_{11}Q_{22} - Q_{12}Q_{21}.$$

So:

$$z_A = \frac{1}{\det(\mathbf{Q})} (Q_{22}h_1 - Q_{12}h_2), \quad z_S = \frac{1}{\det(\mathbf{Q})} (-Q_{21}h_1 + Q_{11}h_2).$$

This makes the computation fully explicit.

2.7.3 F.2.0 Where does the negative log-posterior come from? What does “up to constants” mean?

Step 0: Start from Bayes’ rule (the definition of the posterior). We condition on the observed sensor values (y_A, y_S) and seek the posterior over the latents (z_A, z_S) :

$$p(z_A, z_S \mid y_A, y_S) = \frac{p(y_A, y_S \mid z_A, z_S) p(z_A, z_S)}{p(y_A, y_S)}.$$

The denominator $p(y_A, y_S)$ is a *normalizing constant* with respect to (z_A, z_S) ; it does not depend on (z_A, z_S) .

Step 1: Use the model’s conditional independence structure. Given the two-latent model, the observations satisfy:

$$y_A \perp y_S \mid (z_A, z_S), \quad \text{and} \quad y_A \mid z_A \sim \mathcal{N}(z_A, \sigma_A^2), \quad y_S \mid z_S \sim \mathcal{N}(z_S, \sigma_S^2).$$

So the likelihood factorizes:

$$p(y_A, y_S \mid z_A, z_S) = p(y_A \mid z_A) p(y_S \mid z_S).$$

Step 2: Factorize the prior into components (optional but standard). A common specification is:

$$p(z_A, z_S) = p(z_A) p(z_S) p(z_S - z_A),$$

where:

$$z_A \sim \mathcal{N}(\mu_A, v_A), \quad z_S \sim \mathcal{N}(\mu_S, v_S), \quad (z_S - z_A) \sim \mathcal{N}(\delta, \sigma_C^2).$$

This expresses the coupling as a prior on the *difference* $z_S - z_A$.

Step 3: Write the posterior as a product (dropping the denominator). Combining Steps 0–2:

$$p(z_A, z_S \mid y_A, y_S) \propto p(y_A \mid z_A) p(y_S \mid z_S) p(z_A) p(z_S) p(z_S - z_A),$$

where the symbol “ $\propto$ ” means “equal up to a multiplicative constant independent of (z_A, z_S) ”.

Step 4: Take logs (turn products into sums). Taking log of both sides gives:

$$\log p(z_A, z_S \mid y_A, y_S) = \log p(y_A \mid z_A) + \log p(y_S \mid z_S) + \log p(z_A) + \log p(z_S) + \log p(z_S - z_A) - \log p(y_A, y_S).$$

The last term $\log p(y_A, y_S)$ depends only on data, not on (z_A, z_S) .

Step 5: Define the negative log-posterior objective. We define the *negative log-posterior* as:

$$\mathcal{J}(z_A, z_S) \triangleq -\log p(z_A, z_S \mid y_A, y_S).$$

This is useful because:

$$\arg \max_{z_A, z_S} p(z_A, z_S \mid y_A, y_S) \iff \arg \min_{z_A, z_S} \mathcal{J}(z_A, z_S).$$

What does “up to constants” mean? From the log expression above, any term that does not depend on (z_A, z_S) (e.g. $-\log p(y_A, y_S)$) adds a constant shift to $\mathcal{J}$. Such constants do *not* change:

- the minimizer (MAP estimate), since adding a constant does not change $\arg \min$;
- the curvature / quadratic form in (z_A, z_S) that determines the posterior mean/covariance.

Therefore we write:

$$\mathcal{J}(z_A, z_S) = -\log p(y_A \mid z_A) - \log p(y_S \mid z_S) - \log p(z_A) - \log p(z_S) - \log p(z_S - z_A) + \text{const},$$

and we simply drop the “const” when doing optimization.

Step 6: Convert each Gaussian log-density into a squared-error term. Recall the Gaussian pdf:

$$\mathcal{N}(x \mid m, s^2) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(x - m)^2}{2s^2}\right).$$

Take negative log:

$$-\log \mathcal{N}(x \mid m, s^2) = \frac{(x - m)^2}{2s^2} + \frac{1}{2} \log(2\pi s^2).$$

The second term $\frac{1}{2} \log(2\pi s^2)$ is *constant w.r.t. x* (and w.r.t. the latent variables if s^2 is fixed), so it becomes part of “+const”.

Apply this identity to each factor.

$$\begin{aligned}
-\log p(y_A | z_A) &= -\log \mathcal{N}(y_A | z_A, \sigma_A^2) = \frac{(y_A - z_A)^2}{2\sigma_A^2} + \text{const}, \\
-\log p(y_S | z_S) &= \frac{(y_S - z_S)^2}{2\sigma_S^2} + \text{const}, \\
-\log p(z_A) &= -\log \mathcal{N}(z_A | \mu_A, v_A) = \frac{(z_A - \mu_A)^2}{2v_A} + \text{const}, \\
-\log p(z_S) &= \frac{(z_S - \mu_S)^2}{2v_S} + \text{const}, \\
-\log p(z_S - z_A) &= -\log \mathcal{N}((z_S - z_A) | \delta, \sigma_C^2) = \frac{((z_S - z_A) - \delta)^2}{2\sigma_C^2} + \text{const}.
\end{aligned}$$

Step 7: Sum them to obtain the quadratic objective. Dropping additive constants and multiplying by 2 (which also does not change arg min), we obtain the commonly written objective:

$$\mathcal{J}(z_A, z_S) = \frac{(z_A - \mu_A)^2}{v_A} + \frac{(z_S - \mu_S)^2}{v_S} + \frac{(y_A - z_A)^2}{\sigma_A^2} + \frac{(y_S - z_S)^2}{\sigma_S^2} + \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} \quad (\text{up to constants}).$$

This is exactly the “negative log-posterior (up to constants)” expression used before taking derivatives.

2.7.4 F.3 Relationship between Derivation E and Derivation F

E is a special case of F. If coupling is extremely strong ($\sigma_C^2 \rightarrow 0$), then $(z_S - z_A) \approx \delta$ almost deterministically, so effectively there is one latent:

$$z_A \approx z_S - \delta \equiv z_k,$$

and the model reduces to two measurements of the same latent (Derivation E).

F is more general. If σ_C^2 is moderate/large, it admits that ASV and SV can diverge (different “latents”) while still borrowing strength through the coupling term.

2.7.5 F.4 Practical guidance: how to choose / estimate σ_C^2 and δ

Estimating δ . Same calibration-set mean/median as in Derivation E:

$$\hat{\delta} = \text{median}_{k \in \mathcal{C}}(y_S(k) - y_A(k)), \quad \text{or} \quad \hat{\delta} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).$$

Estimating σ_C^2 (coupling strength). On calibration set $\mathcal{C}$, compute residuals:

$$r_k = (y_S(k) - y_A(k)) - \hat{\delta}.$$

Then:

$$\hat{\sigma}_C^2 = \text{Var}\{r_k : k \in \mathcal{C}\}.$$

Interpretation: if ASV and SV track each other tightly, residual variance is small $\Rightarrow$ strong coupling. If residual variance is large, coupling must be weak.

2.8 Derivation G: From keyword-level posteriors to prompt-level posterior (uncertainty)

If you aggregate *means* linearly on log-scale:

$$z_p = \sum_k w_{p,k} z_k,$$

and assume independent keyword posteriors:

$$z_k \sim \mathcal{N}(\mu_k, v_k), \quad \text{independent across } k,$$

then:

$$z_p \sim \mathcal{N}\left(\sum_k w_{p,k} \mu_k, \sum_k w_{p,k}^2 v_k\right).$$

So:

$$\mu_p = \sum_k w_{p,k} \mu_k, \quad v_p = \sum_k w_{p,k}^2 v_k.$$

Then prompt-level demand $\theta_p = \exp(z_p)$ is lognormal, with:

$$\mathbb{E}[\theta_p] = \exp\left(\mu_p + \frac{v_p}{2}\right), \quad \text{Median}(\theta_p) = \exp(\mu_p).$$

This gives a fully uncertainty-aware prompt-level estimate.

2.9 Derivation H: Why “observed vs calculated vs hyperparameters” matters mathematically

Observed (O). These enter the posterior only through likelihood terms (data constraints).

Calculated (C). These are deterministic functions of observed quantities and hyperparameters (e.g., μ_k and v_k are calculated after you set μ_0, v_0, σ^2 and observe y).

Hyperparameters (H). These control inductive bias and smoothing:

- μ_0, v_0 decide how strongly we believe the prior (regularization).
- σ_A^2, σ_S^2 decide how much we trust each sensor (measurement reliability).
- τ decides how sharply we assign keywords to prompts (semantic precision).
- Coupling terms (δ, σ_C^2) decide how SV and ASV share information.

2.10 End goal: after reading derivations, you should be able to re-derive every formula in Case 1 and Case 2

At this point you have:

1. Closed-form posterior for Case 1 (ASV only).
2. Conversion to original scale and uncertainty intervals.
3. Similarity weights and aggregation math.
4. Closed-form fusion for Case 2 (ASV + SV), including calibration offset δ .
5. Optional coupled-latent variant derivation blueprint.
6. Prompt-level posterior mean/variance derivation.

These are the mathematical “building blocks” used by the implementations/algorithms.

3. Case 1 (Only ASV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)

High-level goal (in plain words)

We want to estimate the monthly AI-search demand for a single user prompt p (example: “*best running shoes for flat feet*”) when we **only** have an AI-demand sensor called ASV($\cdot$). Because demand is positive and varies over orders of magnitude, we do everything on the **log scale** and use **Normal** distributions there.

Core notation and variable-types (Observed vs Calculated vs Hyperparameters)

- **Prompt:** p (a text query).
- **Keywords derived from prompt:** $\{k_i\}_{i=1}^m$.
- **Observed (O):** values returned by sensors / APIs

$\text{ASV}(k_i)$ (AI-search sensor output in proxy units / month).

We will log-transform them:

$$x_i \triangleq \log(\text{ASV}(k_i)).$$

- **Latent unknowns (U):** true AI-demand volumes (unobserved)

$$A^*(k_i) > 0, \quad a_i \triangleq \log(A^*(k_i)).$$

- **Calculated (C):** quantities computed by the algorithm

w_{p,k_i} (prompt→keyword mixture weights), $\mu_{i,\text{post}}, \sigma_{\text{post}}^2$ (posterior params), $\widehat{A}^*(k_i)$, $\widehat{Y}(p)$.

- **Hyperparameters (H):** fixed constants you choose or pre-calibrate

β (softmax sharpness), b_A (ASV log-bias), $\sigma_{A,c}$ (ASV log-noise, for class c), μ_c, τ_c (prior mean/std for

The probabilistic model (the “math story”)

(1) Prior on log-demand. If keyword k_i belongs to a semantic class c (e.g., *running shoes*), we assume

$$a_i \sim \mathcal{N}(\mu_c, \tau_c^2).$$

This says: before seeing any sensor, we believe log-demand is roughly Normal with center μ_c and spread τ_c .

(2) ASV measurement model (likelihood). The ASV sensor is a noisy proxy for $A^*(k_i)$, modeled on log-scale as

$$x_i = \log(\text{ASV}(k_i)) \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2).$$

So: observed log-ASV equals true log-demand plus sensor bias b_A , plus Normal noise.

(3) Conjugate posterior (Normal–Normal). Because both prior and likelihood are Normal in a_i , the posterior is also Normal:

$$a_i \mid x_i \sim \mathcal{N}(\mu_{i,\text{post}}, \sigma_{\text{post}}^2),$$

with

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\tau_c^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\mu_c}{\tau_c^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

Important: σ_{post}^2 is the same for all i in the same class c (because $\tau_c, \sigma_{A,c}$ are shared), but $\mu_{i,\text{post}}$ changes with x_i .

(4) Prompt-level mixture (how keywords combine into one prompt volume). We model the prompt-level AI-demand as a convex combination (mixture) of keyword-level AI-demands:

$$Y(p) \triangleq \sum_{i=1}^m w_{p,k_i} A^*(k_i), \quad \sum_{i=1}^m w_{p,k_i} = 1, \quad w_{p,k_i} \geq 0.$$

The weights w_{p,k_i} come from semantic similarity between p and k_i (micro-steps below).

Running dry-run example (fixed numbers used end-to-end)

p = “best running shoes for flat feet”, $(k_1, k_2, k_3) =$ (“running shoes flat feet”, “stability running shoes”, “arch s

Step 0 (O → C): Write the prompt and pick a small candidate set of keywords. (i)

Motivation. A prompt is broad; we approximate its demand by mixing a few representative keywords.

(ii) Theory. We assume a small set $\{k_i\}$ captures most of the prompt intent-mass.

(iii) Parameters (types). None new. This is a modeling choice, not a statistical parameter.

(iv) Exact calculations. No arithmetic yet.

(v) Dry-run. We choose $m = 3$ keywords listed above.

Step 1 (C): Embed the prompt and keywords into a semantic vector space. (i)

Motivation. We need a numeric way to measure semantic closeness between p and each k_i .

(ii) Theory. Use an embedding model $E(\cdot)$ mapping text $\rightarrow \mathbb{R}^d$, then compare with cosine similarity:

$$s_i \triangleq \cos(E(p), E(k_i)) = \frac{E(p)^\top E(k_i)}{\|E(p)\| \|E(k_i)\|}.$$

Cosine similarity lies in $[-1, 1]$; larger means more similar.

(iii) Parameters (types). The embedding model choice is a **hyperparameter/design choice**.

(iv) Exact calculations. The dot-products/norms are internal to the embedding; we will treat resulting cosine scores as computed outputs.

(v) Dry-run (C values). Suppose the cosine similarities are:

$$(s_1, s_2, s_3) = (0.92, 0.85, 0.74).$$

Step 2 (H → C): Convert similarities into nonnegative unnormalized weights. (i)

Motivation. Cosine scores are not probabilities. We need nonnegative numbers that accentuate higher similarity.

(ii) Theory. Use a softmax with sharpness $\beta > 0$:

$$\tilde{w}_i \triangleq \exp(\beta s_i).$$

Larger β makes the top keyword dominate more.

(iii) **Parameters (types).** β is a **hyperparameter (H)**.

(iv) **Exact calculations.** For the dry-run we set $\beta = 5$ and compute:

$$\tilde{w}_1 = e^{5 \cdot 0.92} = e^{4.60}, \quad \tilde{w}_2 = e^{5 \cdot 0.85} = e^{4.25}, \quad \tilde{w}_3 = e^{5 \cdot 0.74} = e^{3.70}.$$

Now compute each exponential (intermediate arithmetic shown):

$$e^{4.60} \approx 99.484, \quad e^{4.25} \approx 70.105, \quad e^{3.70} \approx 40.447.$$

Sum:

$$\tilde{W} = \tilde{w}_1 + \tilde{w}_2 + \tilde{w}_3 \approx 99.484 + 70.105 + 40.447 = 210.036.$$

(v) **Dry-run.** We have unnormalized weights $\tilde{w}_i$ and total $\tilde{W}$.

Step 3 (C): Normalize to get mixture weights w_{p,k_i} . (i) **Motivation.** We want convex weights that sum to 1.

(ii) **Theory.** Normalize:

$$w_{p,k_i} \triangleq \frac{\tilde{w}_i}{\sum_{j=1}^m \tilde{w}_j}.$$

(iii) **Parameters (types).** No new parameters.

(iv) **Exact calculations (with intermediates).**

$$w_{p,k_1} = \frac{99.484}{210.036} = 0.4737, \quad w_{p,k_2} = \frac{70.105}{210.036} = 0.3337, \quad w_{p,k_3} = \frac{40.447}{210.036} = 0.1926.$$

Check:

$$0.4737 + 0.3337 + 0.1926 = 1.0000 \quad (\text{up to rounding}).$$

(v) **Dry-run.** Final weights:

$$(w_{p,k_1}, w_{p,k_2}, w_{p,k_3}) \approx (0.474, 0.334, 0.193).$$

Step 4 (O): Fetch observed ASV sensor values for each keyword. (i) **Motivation.** These are our only measurements of AI-demand.

(ii) **Theory.** $\text{ASV}(k_i)$ are proxy monthly units. We treat them as noisy observations of $A^*(k_i)$.

(iii) **Parameters (types).** None new; this is observed data.

(iv) **Exact calculations.** No arithmetic yet.

(v) **Dry-run (Observed values).**

$$\text{ASV}(k_1) = 2400, \quad \text{ASV}(k_2) = 1800, \quad \text{ASV}(k_3) = 700.$$

Step 5 (O $\rightarrow$ C): Log-transform observed ASV values. (i) **Motivation.** Demand is multiplicative and heavy-tailed; logs stabilize variance and make Normal modeling plausible.

(ii) **Theory.** Define:

$$x_i \triangleq \log(\text{ASV}(k_i)).$$

(iii) **Parameters (types).** None new.

(iv) **Exact calculations (intermediate shown).**

$$x_1 = \log(2400) = \log(24) + \log(100) \approx 3.178054 + 4.605170 = 7.783224,$$

$$x_2 = \log(1800) = \log(18) + \log(100) \approx 2.890372 + 4.605170 = 7.495542,$$

$$x_3 = \log(700) = \log(7) + \log(100) \approx 1.945910 + 4.605170 = 6.551080.$$

(v) **Dry-run.**

$$(x_1, x_2, x_3) \approx (7.783, 7.496, 6.551).$$

Step 6 (H): Choose / state the class-conditional prior hyperparameters (μ_c, τ_c) and sensor hyperparameters $(b_A, \sigma_{A,c})$. (i) **Motivation.** Bayesian inference needs a prior and a likelihood noise model.

(ii) **Theory.** Prior: $a_i \sim \mathcal{N}(\mu_c, \tau_c^2)$, likelihood: $x_i | a_i \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2)$.

Class- c hyperparameters (estimated, not assumed). For each intent class c with keyword set $K_c = \{k : c(k) = c\}$ and historical window $t = 1, \dots, T$:

μ_c, τ_c^2 (prior mean/variance of latent log-demand within class c),

b_A or $b_{A,c}$ (ASV log-bias), $\sigma_{A,c}^2$ (ASV measurement-noise variance in class c).

These quantities are computed from historical sensor time series $\text{ASV}_{k,t}$ via the explicit estimators in Derivation 2.5.

Estimate class- c hyperparameters from historical ASV series (Empirical Bayes).

Given $\text{ASV}_{k,t}$, define $x_{k,t} = \log(\text{ASV}_{k,t})$. Compute:

$$\bar{x}_k = \frac{1}{T} \sum_{t=1}^T x_{k,t}, \quad \hat{\sigma}_A^2(k) = \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2, \quad \hat{\sigma}_{A,c}^2 = \frac{1}{|K_c|} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

If a calibration proxy $g_{k,t} \approx \log(A_{k,t}^*)$ exists on a calibration set $\mathcal{C}$, estimate bias:

$$\hat{b}_A = \frac{1}{|\mathcal{C}|} \sum_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

Then compute de-biased keyword level estimates:

$$\hat{a}_k = \bar{x}_k - \hat{b}_A, \quad \hat{\mu}_c = \frac{1}{|K_c|} \sum_{k \in K_c} \hat{a}_k,$$

and the between-keyword variance with averaging-noise removal:

$$\hat{\tau}_c^2 = \max \left\{ \frac{1}{|K_c| - 1} \sum_{k \in K_c} (\hat{a}_k - \hat{\mu}_c)^2 - \frac{\hat{\sigma}_{A,c}^2}{T}, \epsilon \right\}.$$

Use $(\hat{\mu}_c, \hat{\tau}_c, \hat{b}_A, \hat{\sigma}_{A,c})$ in the subsequent Case 1 posterior steps.

(iii) **Parameters (types).**

$\mu_c, \tau_c, b_A, \sigma_{A,c}$ are all (estimated) hyperparameters (H) as estimated above.

(iv) **Exact values used in dry-run.**

$$\mu_c = \log(1200) = \log(12) + \log(100) \approx 2.484907 + 4.605170 = 7.090077,$$

$$\tau_c = 0.60, \quad b_A = 0, \quad \sigma_{A,c} = 0.35.$$

(v) **Dry-run.** Keep these fixed for all three keywords (same class c).

Step 7 (C): Compute the posterior variance σ_{post}^2 on the log-scale (same for all keywords in this class). (i) **Motivation.** Posterior variance tells us how uncertain a_i remains after seeing the sensor.

(ii) **Theory.**

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\tau_c^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}.$$

(iii) **Parameters (types).** Uses hyperparameters $\tau_c, \sigma_{A,c}$.

(iv) **Exact calculations (with all intermediates).** First compute squares:

$$\tau_c^2 = (0.60)^2 = 0.36, \quad \sigma_{A,c}^2 = (0.35)^2 = 0.1225.$$

Compute reciprocals:

$$\frac{1}{\tau_c^2} = \frac{1}{0.36} = 2.777777\dots, \quad \frac{1}{\sigma_{A,c}^2} = \frac{1}{0.1225} = 8.163265\dots$$

Sum:

$$2.777777\dots + 8.163265\dots = 10.941043\dots$$

Invert:

$$\sigma_{\text{post}}^2 = \frac{1}{10.941043\dots} = 0.091402\dots$$

Standard deviation:

$$\sigma_{\text{post}} = \sqrt{0.091402\dots} = 0.302327\dots$$

(v) **Dry-run.** $\sigma_{\text{post}} \approx 0.302$ (same for $i = 1, 2, 3$).

Step 8 (C): Compute each posterior mean $\mu_{i,\text{post}}$ for $a_i = \log(A^*(k_i))$. (i) **Motivation.** The posterior mean on log-scale is our best log-demand estimate under squared-error loss.

(ii) **Theory.**

$$\mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\mu_c}{\tau_c^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

(iii) **Parameters (types).** Uses $\mu_c, \tau_c, b_A, \sigma_{A,c}$ (H) and x_i (C from observed ASV).

(iv) **Exact calculations (show all intermediates).**

Compute common pieces:

$$\frac{\mu_c}{\tau_c^2} = \frac{7.090077}{0.36} = 19.694658, \quad \frac{1}{\sigma_{A,c}^2} = \frac{1}{0.1225} = 8.163265.$$

Now do each i :

$i = 1$:

$$\frac{x_1 - b_A}{\sigma_{A,c}^2} = \frac{7.783224 - 0}{0.1225} = 63.536531,$$

$$\left(\frac{\mu_c}{\tau_c^2} + \frac{x_1 - b_A}{\sigma_{A,c}^2} \right) = 19.694658 + 63.536531 = 83.231189,$$

$$\mu_{1,\text{post}} = \sigma_{\text{post}}^2 \cdot 83.231189 = 0.091402 \cdot 83.231189 = 7.607147.$$

$i = 2$:

$$\frac{x_2 - b_A}{\sigma_{A,c}^2} = \frac{7.495542}{0.1225} = 61.186468,$$

$$19.694658 + 61.186468 = 80.881126,$$

$$\mu_{2,\text{post}} = 0.091402 \cdot 80.881126 = 7.392153.$$

$i = 3$:

$$\frac{x_3 - b_A}{\sigma_{A,c}^2} = \frac{6.551080}{0.1225} = 53.477388,$$

$$19.694658 + 53.477388 = 73.172046,$$

$$\mu_{3,\text{post}} = 0.091402 \cdot 73.172046 = 6.688654.$$

(v) **Dry-run.**

$$(\mu_{1,\text{post}}, \mu_{2,\text{post}}, \mu_{3,\text{post}}) \approx (7.607, 7.392, 6.689), \quad \sigma_{\text{post}} \approx 0.302.$$

Step 9 (C): Convert posterior on log-scale back to the original demand scale. (i)

Motivation. Stakeholders care about demand in “units/month”, not log-units.

(ii) Theory (very important subtlety). If $a \sim \mathcal{N}(\mu, \sigma^2)$, then $A = \exp(a)$ is **lognormal**. Two common point summaries:

$$\text{Posterior median: } \text{median}[A] = e^\mu, \quad \text{Posterior mean: } \mathbb{E}[A] = e^{\mu + \frac{1}{2}\sigma^2}.$$

We will compute both explicitly (no hand-waving).

(iii) Parameters (types). Uses computed $(\mu_{i,\text{post}}, \sigma_{\text{post}}^2)$.

(iv) Exact calculations. First compute the correction term:

$$\frac{1}{2}\sigma_{\text{post}}^2 = \frac{1}{2} \cdot 0.091402 = 0.045701.$$

Now for each keyword:

k_1 :

$$\text{median } \widehat{A}_{\text{med}}^*(k_1) = e^{7.607147} = 2012.6,$$

$$\text{mean } \widehat{A}_{\text{mean}}^*(k_1) = e^{7.607147+0.045701} = e^{7.652848} = 2106.7.$$

k_2 :

$$\widehat{A}_{\text{med}}^*(k_2) = e^{7.392153} = 1623.4, \quad \widehat{A}_{\text{mean}}^*(k_2) = e^{7.437854} = 1699.3.$$

k_3 :

$$\widehat{A}_{\text{med}}^*(k_3) = e^{6.688654} = 802.2, \quad \widehat{A}_{\text{mean}}^*(k_3) = e^{6.734355} = 839.5.$$

(v) **Dry-run.** Keyword-level AI demand is roughly:

$$(\widehat{A}_{\text{med}}^*(k_1), \widehat{A}_{\text{med}}^*(k_2), \widehat{A}_{\text{med}}^*(k_3)) \approx (2013, 1623, 802).$$

Step 10 (C): Compute prompt-level AI-demand estimate $\widehat{Y}(p)$ by mixing keyword demands. (i) Motivation. The prompt is a blend of keyword-intents; we aggregate accordingly.

(ii) Theory.

$$\widehat{Y}(p) = \sum_{i=1}^m w_{p,k_i} \widehat{A}^*(k_i).$$

We will compute it using the posterior medians (also called the “typical” value).

(iii) Parameters (types). Uses calculated weights and posterior summaries.

(iv) Exact calculations (show all multiplications). Using

$$(w_1, w_2, w_3) = (0.4737, 0.3337, 0.1926), \quad (A_1, A_2, A_3) = (2012.6, 1623.4, 802.2),$$

we compute:

$$w_1 A_1 = 0.4737 \cdot 2012.6 = 953.9,$$

$$w_2 A_2 = 0.3337 \cdot 1623.4 = 541.7,$$

$$w_3 A_3 = 0.1926 \cdot 802.2 = 154.6.$$

Sum:

$$\hat{Y}_{\text{med}}(p) = 953.9 + 541.7 + 154.6 = 1650.2.$$

(If you prefer using posterior means on original scale, you would replace A_i with $\hat{A}_{\text{mean}}^{\star}(k_i)$; the exact same arithmetic applies.) **(v) Dry-run result.**

$\hat{Y}_{\text{med}}(p) \approx 1650 \text{ AI-units/month}$

Step 11 (C): Compute an explicit uncertainty interval for the prompt-level estimate (no “just sample it” hand-wave). **(i) Motivation.** Any estimate should come with error bars.

(ii) Theory. If $A_i = \exp(a_i)$ with $a_i \sim \mathcal{N}(\mu_i, \sigma^2)$, then

$$\mathbb{E}[A_i] = e^{\mu_i + \frac{1}{2}\sigma^2}, \quad \text{Var}(A_i) = (e^{\sigma^2} - 1) e^{2\mu_i + \sigma^2}.$$

Assuming keyword posteriors are approximately independent,¹

$$\mathbb{E}[Y] = \sum_i w_i \mathbb{E}[A_i], \quad \text{Var}(Y) = \sum_i w_i^2 \text{Var}(A_i).$$

Then an approximate 90% interval (Normal approximation) is

$$\mathbb{E}[Y] \pm 1.645 \cdot \sqrt{\text{Var}(Y)}.$$

(iii) Parameters (types). Uses $(\mu_{i,\text{post}}, \sigma_{\text{post}}^2)$ (C) and w_i (C).

(iv) Exact calculations. We already have $\sigma_{\text{post}}^2 = 0.091402$, so

$$e^{\sigma_{\text{post}}^2} = e^{0.091402} = 1.095726, \quad (e^{\sigma_{\text{post}}^2} - 1) = 0.095726.$$

Compute each $\mathbb{E}[A_i]$ (already computed in Step 9) and $\text{Var}(A_i)$:

$i = 1$:

$$\mathbb{E}[A_1] = 2106.7, \quad \text{Var}(A_1) = 0.095726 \cdot e^{2 \cdot 7.607147 + 0.091402} = 0.095726 \cdot e^{15.305696} = 0.095726 \cdot 4.4239 \times 10^6 = 423,600$$

$i = 2$:

$$\mathbb{E}[A_2] = 1699.3, \quad \text{Var}(A_2) = 0.095726 \cdot e^{2 \cdot 7.392153 + 0.091402} = 0.095726 \cdot e^{14.875708} = 0.095726 \cdot 2.8906 \times 10^6 = 276,600$$

$i = 3$:

$$\mathbb{E}[A_3] = 839.5, \quad \text{Var}(A_3) = 0.095726 \cdot e^{2 \cdot 6.688654 + 0.091402} = 0.095726 \cdot e^{13.468710} = 0.095726 \cdot 7.0713 \times 10^5 = 67,700.$$

Now compute $\mathbb{E}[Y]$:

$$\mathbb{E}[Y] = 0.4737 \cdot 2106.7 + 0.3337 \cdot 1699.3 + 0.1926 \cdot 839.5 = 998.7 + 566.9 + 161.7 = 1727.3.$$

Compute $\text{Var}(Y)$:

$$\text{Var}(Y) = (0.4737)^2 \cdot 423,600 + (0.3337)^2 \cdot 276,600 + (0.1926)^2 \cdot 67,700.$$

¹This is a modeling approximation; if keywords share sensor noise or class-level random effects, covariance terms should be added explicitly.

Compute each term:

$$(0.4737)^2 = 0.2244 \Rightarrow 0.2244 \cdot 423,600 = 95,000,$$

$$(0.3337)^2 = 0.1114 \Rightarrow 0.1114 \cdot 276,600 = 30,800,$$

$$(0.1926)^2 = 0.0371 \Rightarrow 0.0371 \cdot 67,700 = 2,510.$$

Sum:

$$\text{Var}(Y) = 95,000 + 30,800 + 2,510 = 128,310, \quad \sqrt{\text{Var}(Y)} = 358.2.$$

Finally the 90% interval:

$$1727.3 \pm 1.645 \cdot 358.2 = 1727.3 \pm 589.3 \Rightarrow [1138.0, 2316.6].$$

(v) Dry-run interval.

$\hat{Y}(p) \approx 1650 \text{ (median); } 90\% \text{ approx interval } [1138, 2317].$

Step 12 (C): Final algorithm summary (one compact “recipe”). (i) **Motivation.** A short checklist to implement correctly.

(ii) **Theory.** The entire Case 1 pipeline is: similarity $\rightarrow$ weights; ASV $\rightarrow$ posterior on a_i ; mix back on original scale.

(iii) **Parameters (types).** $\beta, \mu_c, \tau_c, b_A, \sigma_{A,c}$ are (H).

(iv) **Exact recipe (math).**

1. Compute cosine similarities $s_i = \cos(E(p), E(k_i))$.
2. Compute weights $w_i = \exp(\beta s_i) / \sum_j \exp(\beta s_j)$.
3. Observe ASV and compute $x_i = \log(\text{ASV}(k_i))$.
4. Compute $\sigma_{\text{post}}^2 = (1/\tau_c^2 + 1/\sigma_{A,c}^2)^{-1}$.
5. Compute $\mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\mu_c/\tau_c^2 + (x_i - b_A)/\sigma_{A,c}^2 \right)$.
6. Convert: median demand $\hat{A}^\star(k_i) = \exp(\mu_{i,\text{post}})$ (or mean $\exp(\mu_{i,\text{post}} + \frac{1}{2}\sigma_{\text{post}}^2)$).
7. Prompt estimate: $\hat{Y}(p) = \sum_i w_i \hat{A}^\star(k_i)$.

(v) Dry-run. We computed $\hat{Y}(p) \approx 1650$ AI-units/month.